CSci 21 Data Structures and Algorithms
Module 7

Introduction to Trees

Learning Outcomes

LO 7.1 Identify valid terminologies and properties of a binary tree

LO 7.2 Perform preorder, inorder, postorder traversals given binary tree definition

LO 7.3 Synthesize a valid postfix expression from a binary expression tree

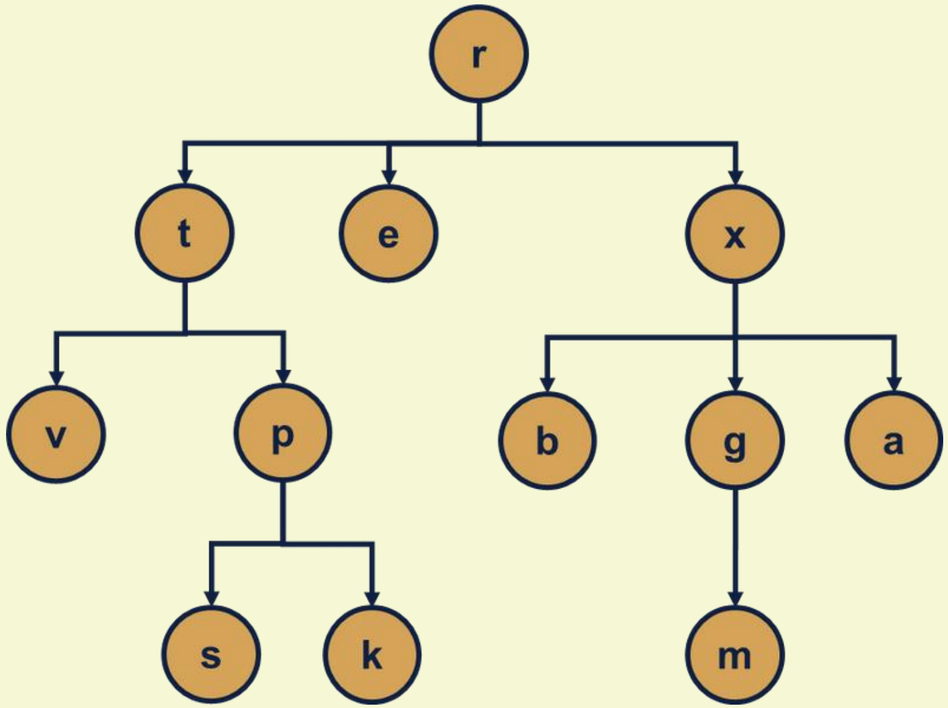
Setting the Expectations

PREREQUISITES

- **Familiarized** the **linked lists, stack, queue, pointer,** and **dynamic memory management in C/C++**—allocation, deallocation, dereferencing, and redirection.

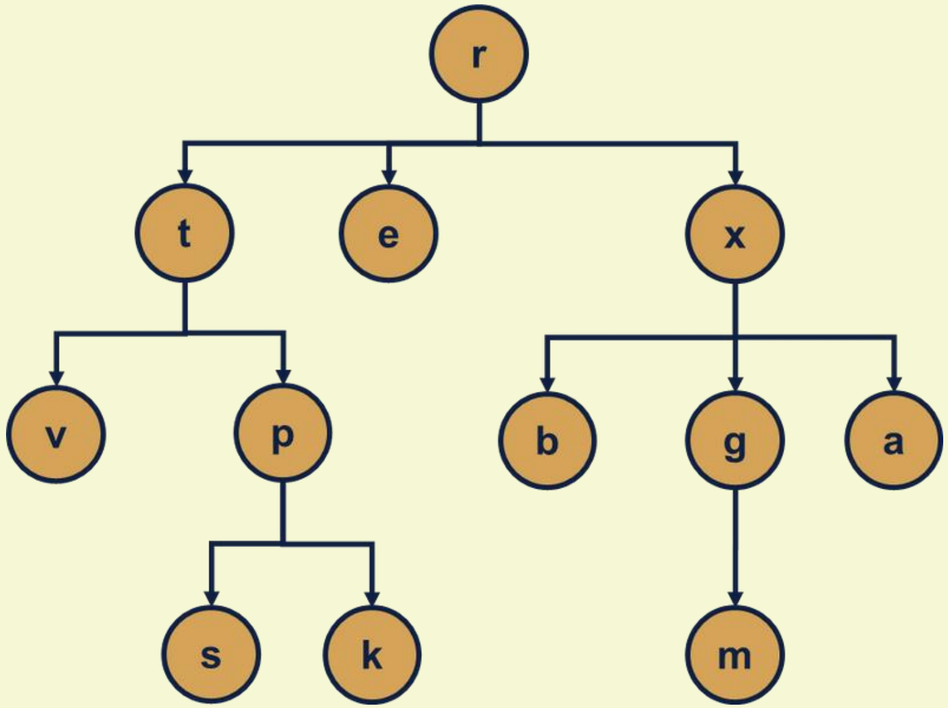
What is a Tree?

A tree is an ADT that consists of a finite set of elements, called tree **nodes**, and a finite set of directed lines, called **branches**, that connect to the nodes.



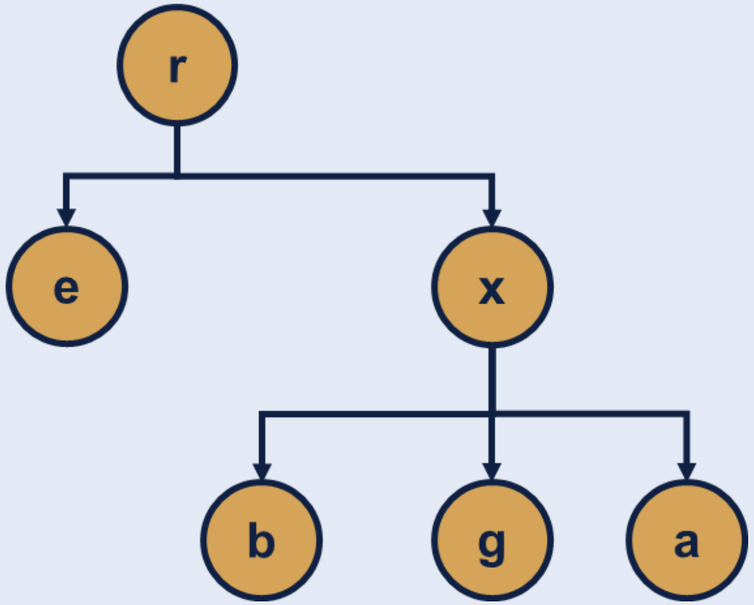
What is a Tree?

Tree, similar to linked lists, is also used as a data structure for storage but is *non-linear* in form.



Tree Terminologies

The number of branches associated with a tree node is the **degree** of the tree node.

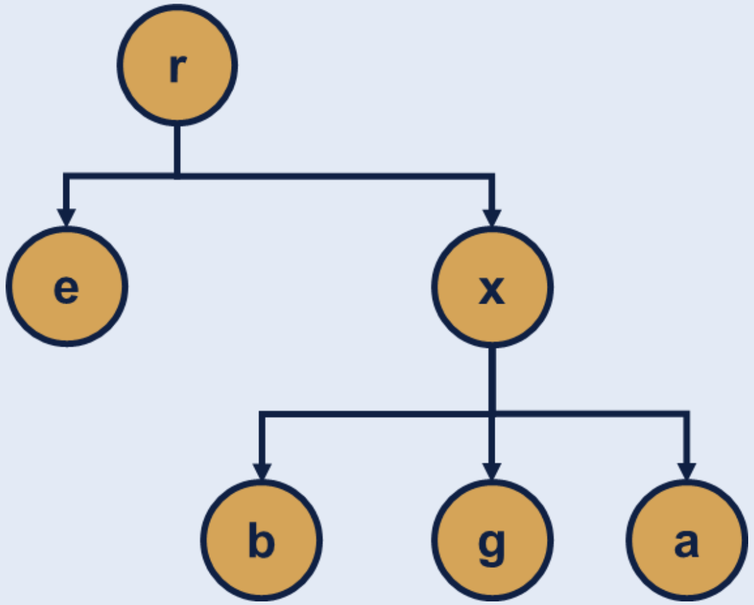


*The tree node 'x' has a **degree** of 4.*

Tree Terminologies

When the branch is directed toward the tree node, it is an **indegree** branch.

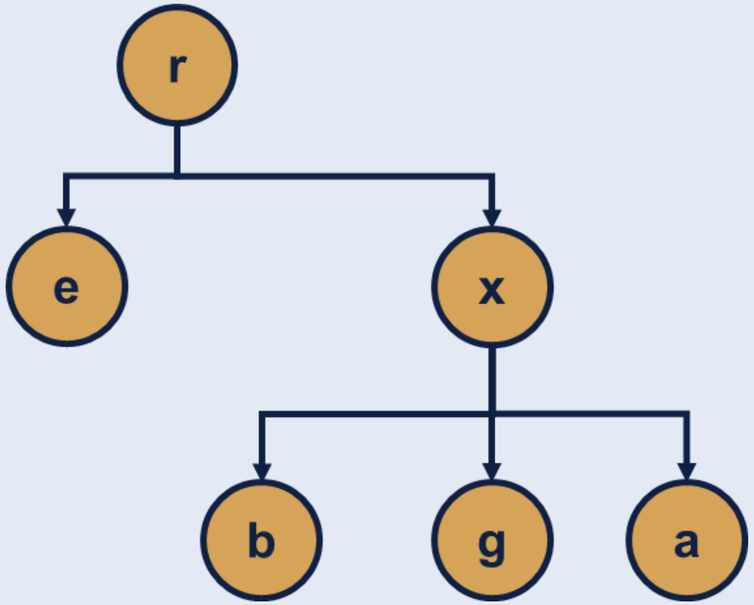
When the branch is directed away from the tree node, it is an **outdegree** branch.



*The tree node 'x' has a **degree** of 4. It has an **indegree** of 1 and an **outdegree** of 3.*

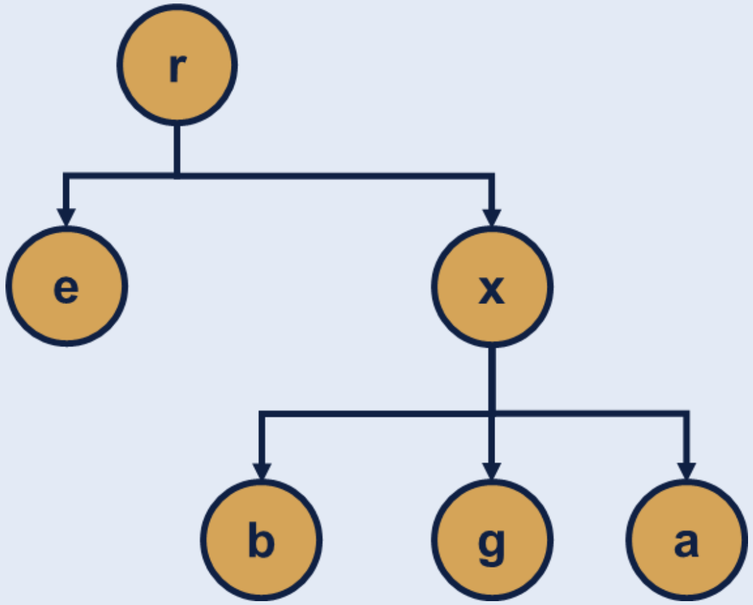
Tree Terminologies

The *sum of the indegree and outdegree* branches is the **degree** of the node.



*The tree node 'x' has a **degree** of 4. It has an **indegree** of 1 and an **outdegree** of 3.*

Tree Terminologies

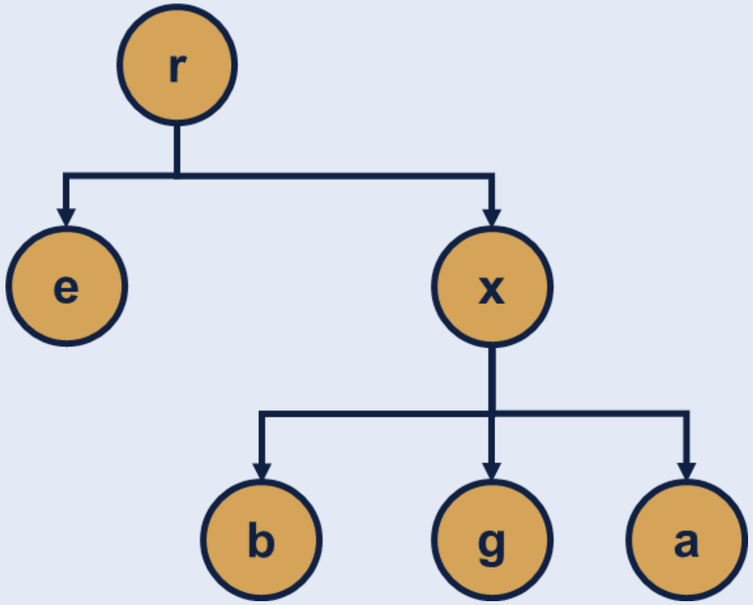


*The tree node 'r' is the **root** of the tree above.*

A **root** of a tree is a tree node in a specific subtree with an indegree of 0.

With the exception of the root, all of the nodes in a tree must have an indegree of *exactly one*.

Tree Terminologies

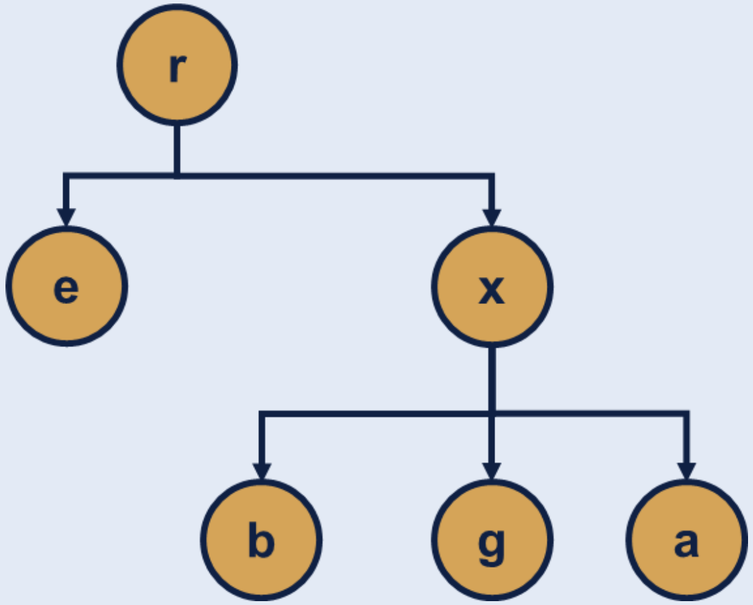


*Tree nodes 'e', 'b', 'g', and 'a' are **leaf nodes** while 'x' is an **internal node**.*

A **leaf** is any tree node with an outdegree of zero, a node with no successors.

A tree node that is *not a root nor a leaf* is known as an **internal node**.

Tree Terminologies

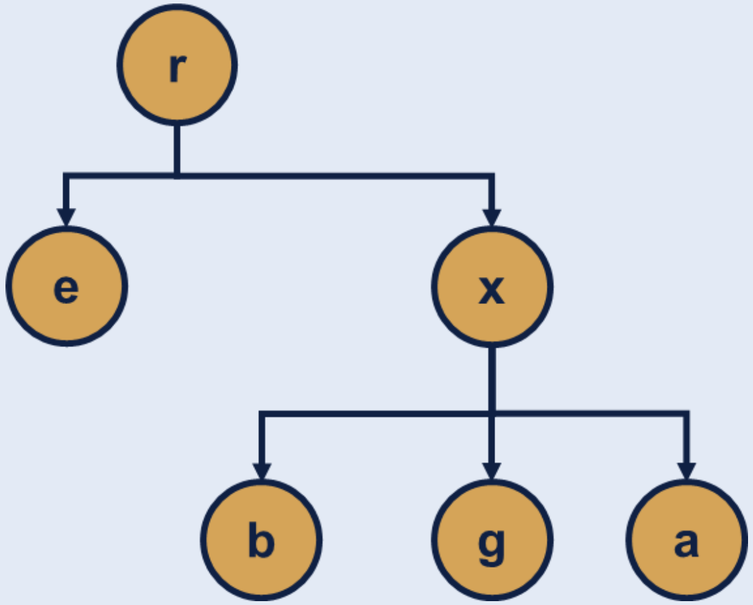


*Tree node 'x' is a **parent** node of 'a' and at the same time a **child** node of 'r'.*

Tree node A is a **parent** node of tree node B if the outgoing branch of A directs as an ingoing branch of B.

With the similar definition, B is a **child** node of A.

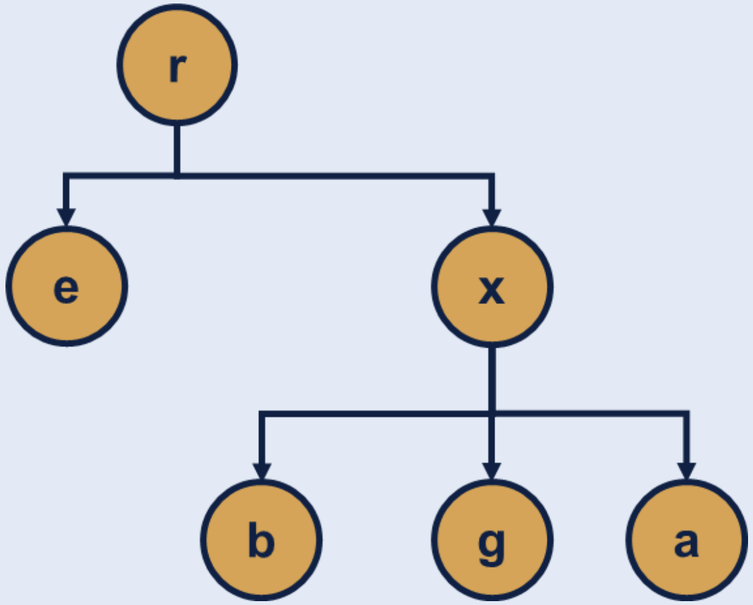
Tree Terminologies



*The tree nodes 'e' and 'x' are **siblings**.*

Two or more tree nodes with the same parent node are **siblings.**

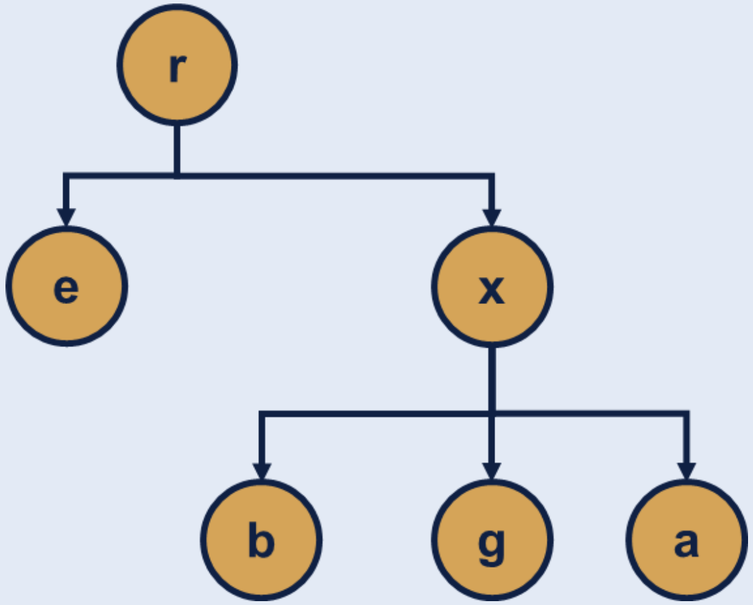
Tree Terminologies



*The **path** from tree node 'r' to 'a' is **r - x - a***

A **path** is a sequence of nodes along the branches of a tree.

Tree Terminologies

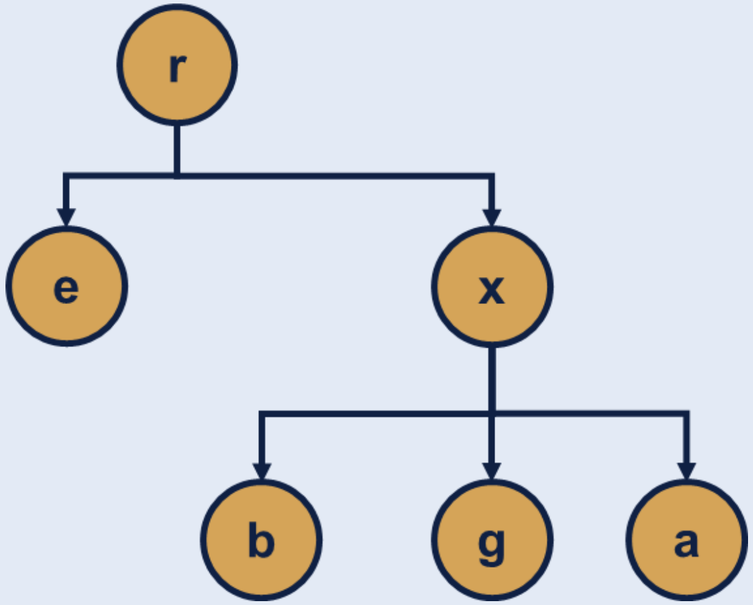


*Based on the **path** from tree node 'r' to 'a': r – x – a, 'r' is an **ancestor** of 'a', and 'x' is a **descendent** of 'r'.*

An **ancestor** of a tree node is any tree node in the path from the root to that tree node.

A **descendent** of a tree node is any tree node in the path from that tree node to any leaf node below (*outgoing*) that tree node.

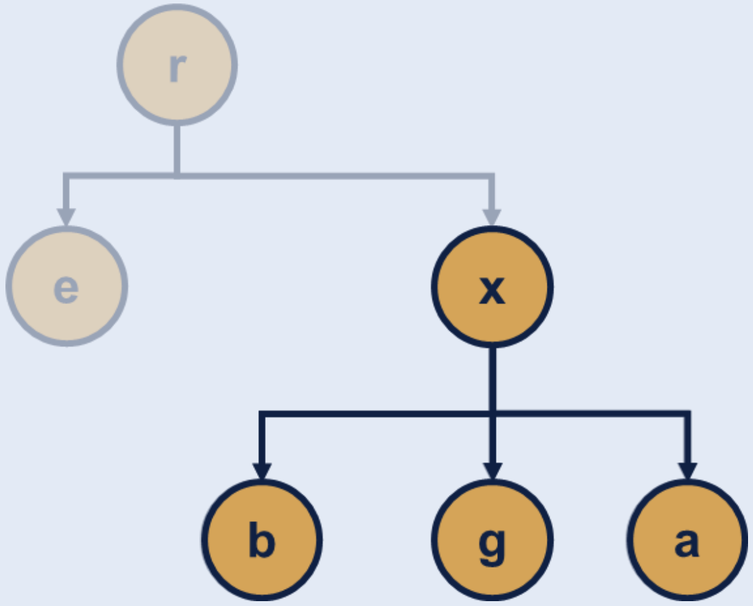
Tree Terminologies



*Tree node 'g' has a **level** of 2.*

A **level** of a tree node is the number of branches it takes to traverse from the root to that tree node. The *root*, by definition, has a *level* of 0.

Tree Terminologies



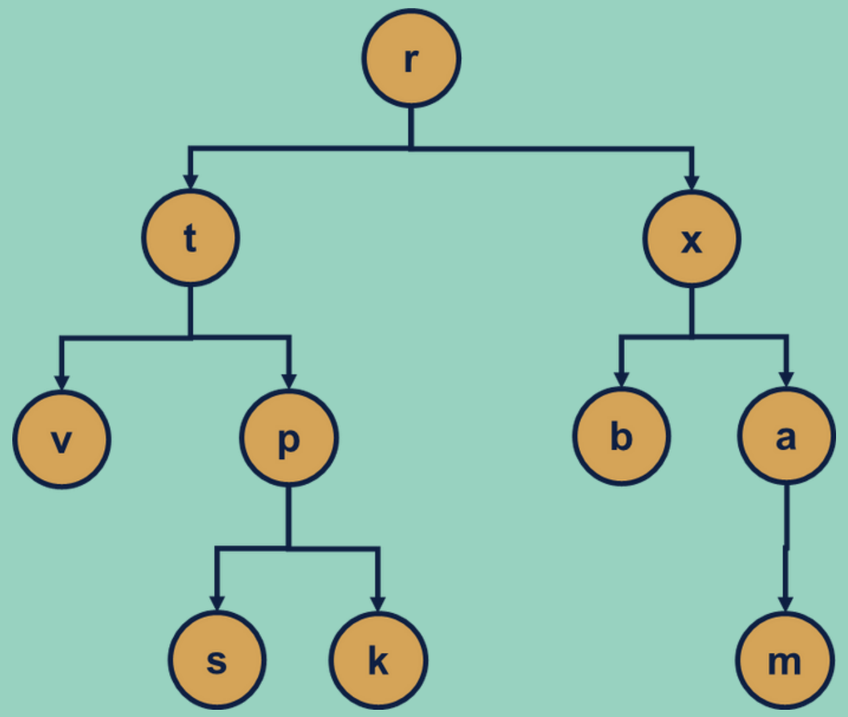
*The tree above is a **subtree** from the tree 'r'. The **height** of this subtree is 2, while the **height** of the previous tree is 3.*

A **subtree** is any connected structure below the root. A subtree based from any tree node makes that tree node the root of that subtree.

The **height** of a tree/subtree is the highest leaf node level plus 1.

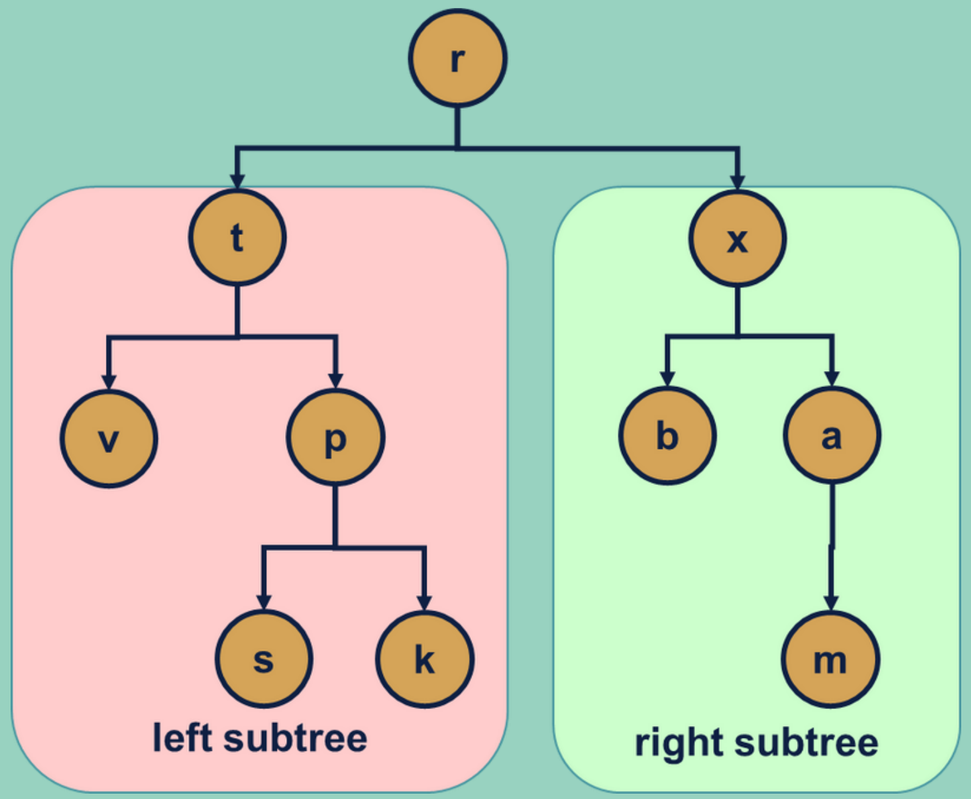
Binary Tree

A **binary tree** is a tree in which no tree node can have more than two *subtrees*; the maximum outdegree for any tree node is **2.**



Binary Tree

These subtrees are designated as the **left subtree** and **right subtree**. Note that each *subtree* itself *is a binary tree*.



Binary Tree Parenthetical Notation

Most binary trees are written in the form a diagram representation but it can also be represented as a text through **parenthetical notations**.

The parenthetical notation follows a recursive grammar:

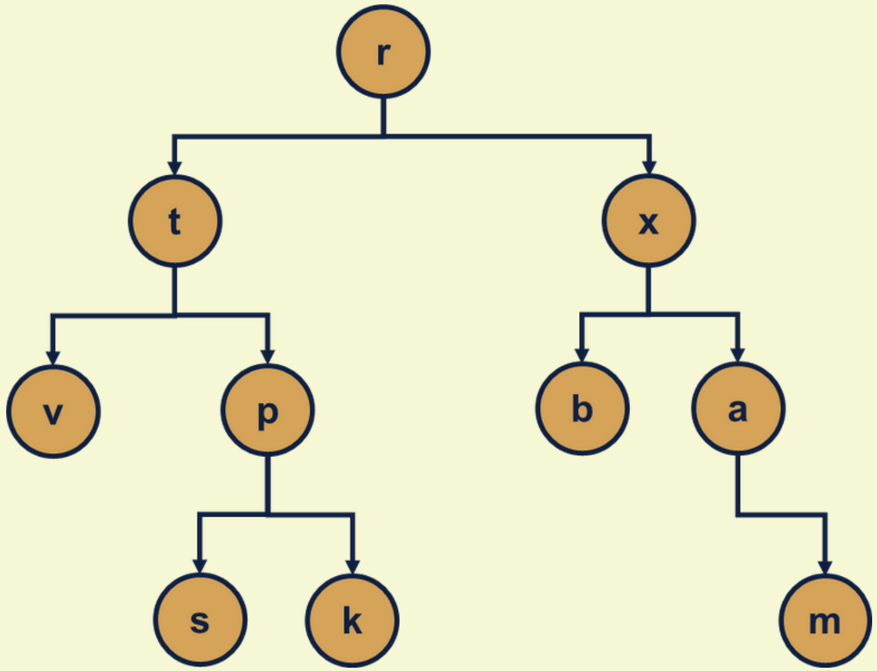
<parent node data> [(**<left child node data><right child node data>**)]

Binary Tree Parenthetical Notation

The brackets ('[' and ']') means it is optional, i.e. in the case of leaf nodes where the node has no child nodes. If in any case there is no left child of a parent node, a symbol '-' is written while if there is a left child but no right child, only the left child node data is written. The delimiter used will be space symbol.

<parent node data> [(<left child node data><right child node data>)]

Binary Tree Parenthetical Notation



The parenthetical notation of the tree on the left is:

r (t (v p (s k)) x (b (e) a (- m)))

Binary Tree Properties

The maximum height of a binary tree with N tree nodes,

$$H_{max} = N$$

The minimum height of a binary tree with N tree nodes,

$$H_{min} = \lfloor \log_2 N \rfloor + 1$$

Binary Tree Properties

The maximum number of nodes of a binary tree with height H,

$$N_{max} = 2^H - 1$$

The minimum number of nodes of a binary tree with height H,

$$N_{min} = H$$

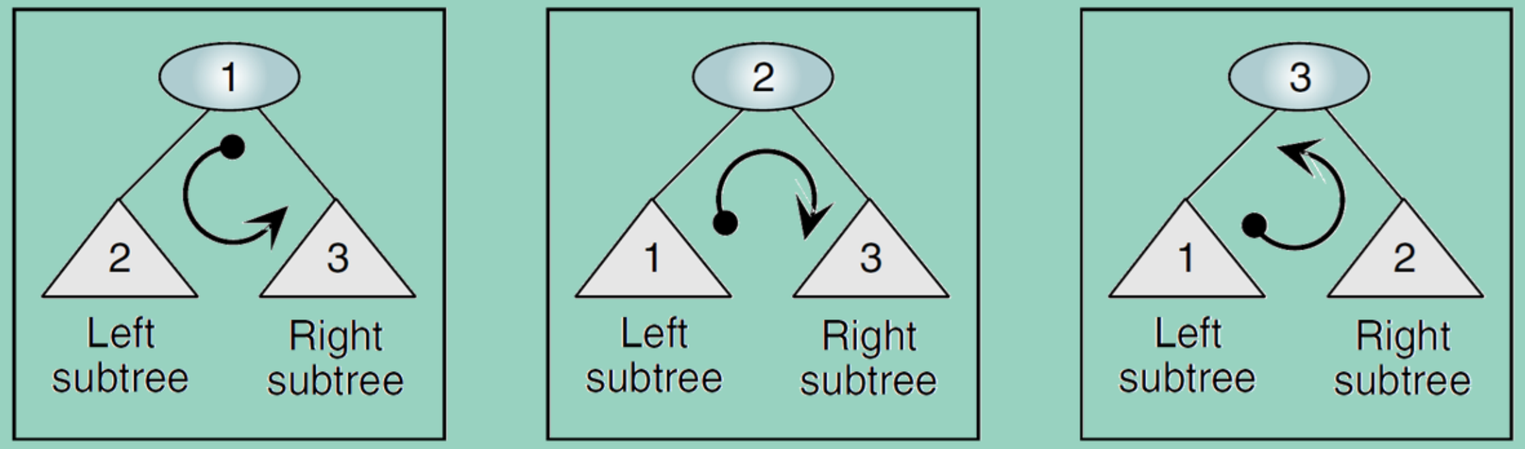
Binary Tree Traversals

A binary tree traversal requires that each node of the tree be processed *once and only once* in a predetermined sequence.

The two general approaches to the traversal sequence are by **depth-first** and **breadth-first**.

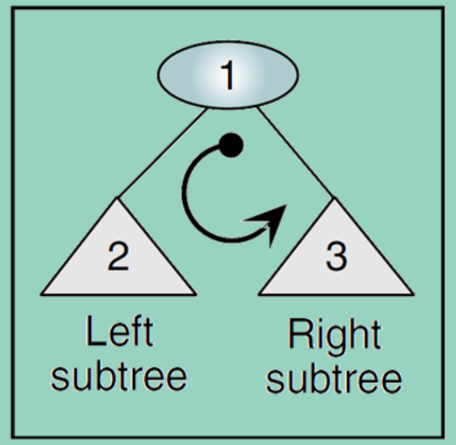
Binary Tree Traversals

In **depth-first traversal**, we process all of the descendents of a child before going on to the next child.



(a) Preorder traversal (b) Inorder traversal (c) Postorder traversal

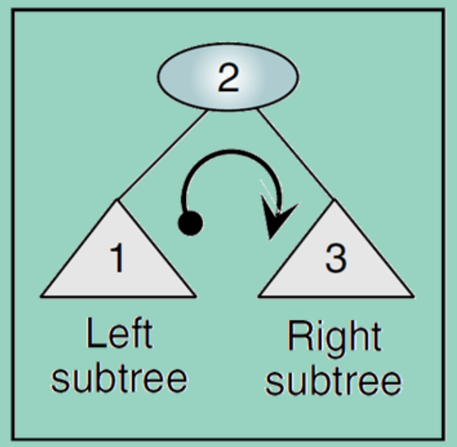
Binary Tree Traversals



(a) Preorder traversal

```
algorithm preOrder (root)
  /*Traverse a binary tree in node-left-right sequence.
  Pre: root is the entry node of a tree or subtree.
  Post: each node has been processed in order.*/
  if (root is not null)
    process (root)
    preOrder (root's left child)
    preOrder (root's right child)
  end if
end preOrder
```

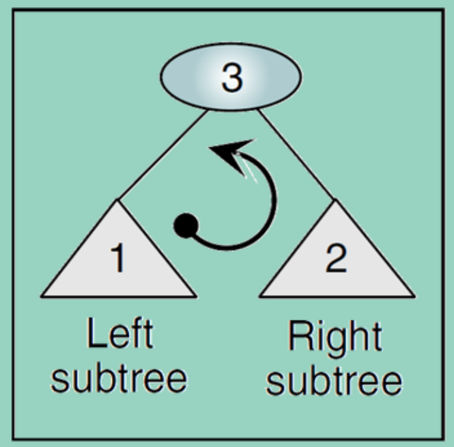
Binary Tree Traversals



(b) Inorder traversal

```
algorithm inOrder (root)
  /*Traverse a binary tree in left-node-right sequence.
  Pre: root is the entry node of a tree or subtree.
  Post: each node has been processed in order.*/
  if (root is not null)
    inOrder (root's left child)
    process (root)
    inOrder (root's right child)
  end if
end inOrder
```

Binary Tree Traversals

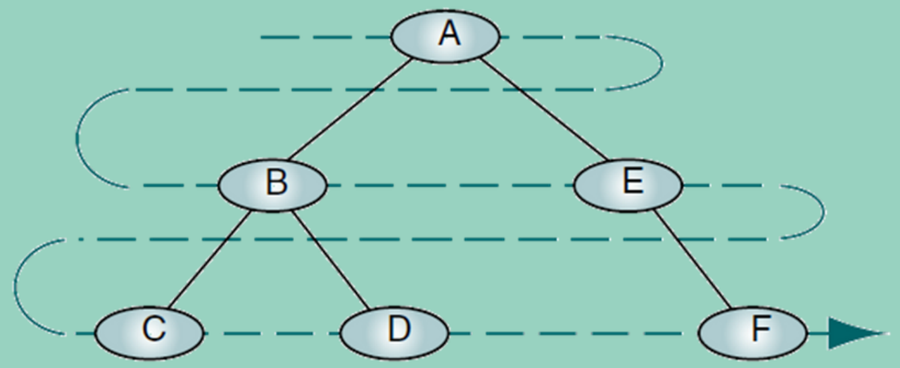


(c) Postorder traversal

```
algorithm postOrder (root)
  /*Traverse a binary tree in left-right-node sequence.
  Pre: root is the entry node of a tree or subtree.
  Post: each node has been processed in order.*/
  if (root is not null)
    postOrder (root's left child)
    postOrder (root's right child)
    process (root)
  end if
end postOrder
```

Binary Tree Traversals

In **breadth-first traversal**, the processing proceeds horizontally *from the root to all of its children, then to its children's children*, and so on until all tree nodes have been processed.



Binary Expression Trees

An **expression** is a sequence of tokens that follow prescribed rules.

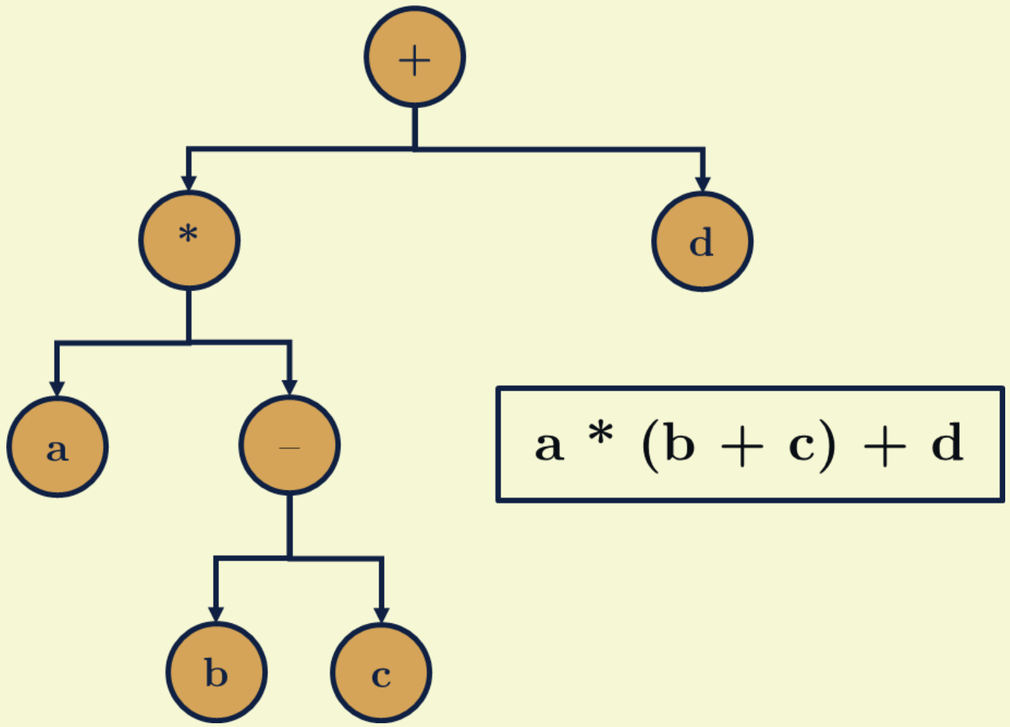
A **token** may be either an operand or an operator.

In this discussion, we consider only binary arithmetic operators in the form *operand-operator-operand*.

Binary Expression Trees

An **expression tree** is a binary tree with the following properties:

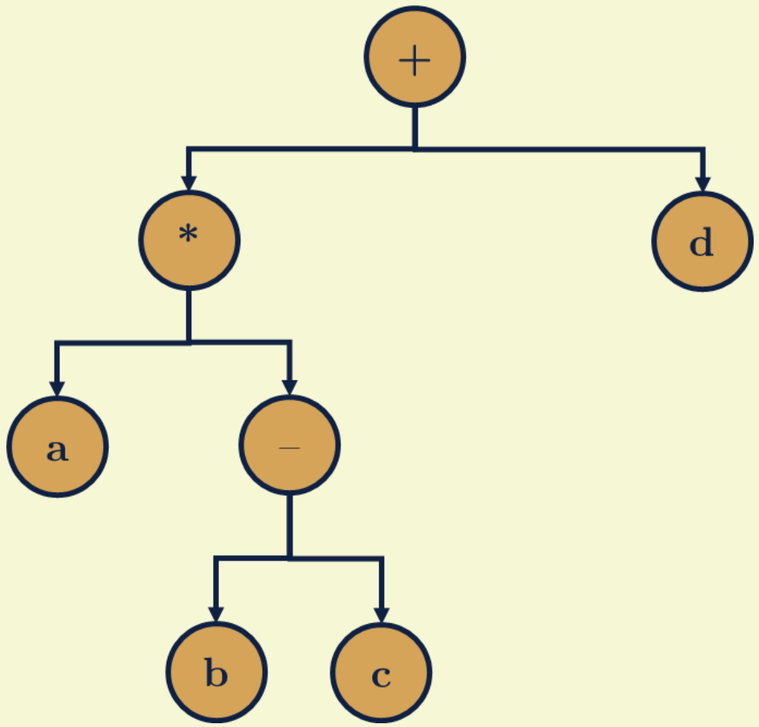
1. *Each leaf is an operand.*
2. *The root and internal nodes are operators.*
3. *Subtrees are subexpressions, with the root being an operator.*



Binary Expression Trees

For this expression tree, the post-fix expression (*through post-order traversal*) is,

a b c - * d +



"The key to wisdom is constant and frequent questioning, for by doubting we are led to question, by questioning we arrive at the truth..."

- Peter Abelard

OPEN FORUM Q/A

for Module 7